

Overview

The program matches sample DNA sequences against virus signature sequences and calculates a match score for each pair. It uses a sliding window approach to compare substrings of a sample sequence with a signature sequence. A match is determined based on character equality, allowing for 'N' as a wildcard. The match score is calculated as the average Phred+33 quality score of matching characters. Additionally, an integrity hash is computed for each sample based on its quality scores.

Our implementation uses two CUDA kernels:

1. `matchKernel`: Computes match scores for all sample-signature pairs.
2. `integrityHashKernel`: Computes integrity hashes for all samples.

For `matchKernel`, a 2D grid is used where the number of blocks in the x and y dimensions corresponds to the number of samples and signatures respectively. Each block represents a sample-signature pair and contains a specified number of threads. Threads within the block process different sliding window positions in the sample sequence in parallel.

For `integrityHashKernel`, a 1D grid is used with enough blocks to cover all samples, and each block contains a specified number of threads. Each thread processes one sample to compute its hash.

The grid and block dimensions are chosen to maximize parallelism while ensuring each thread has sufficient work. The 2D grid in `matchKernel` allows for independent processing of sample-signature pairs while the 1D grid in `integrityHashKernel` simplifies the computation of hashes.

Input data (sequences, quality scores, pointer offsets) and output data (match scores, integrity hashes) are stored in global memory. Data is transferred between host and device using `cudaMemcpy`. In `matchKernel`, shared memory is used to store the best match score for each thread within a block. This reduces global memory access to minimise latency and allows threads to collaboratively determine the best score for the block. Synchronization (`__syncthreads()`) ensures correctness when threads share data.

Performance Analysis

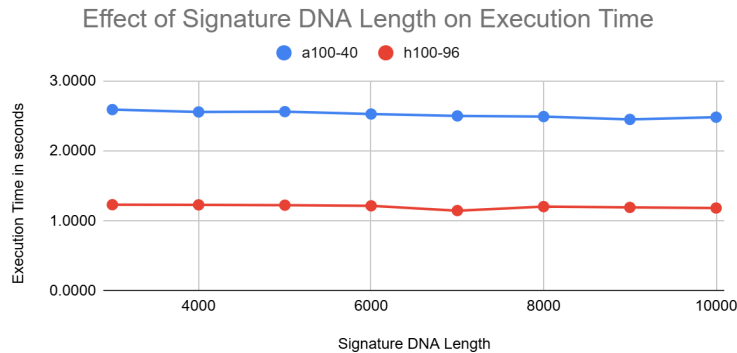
Sample DNA Length

As the sample DNA length increases, the number of sliding window positions and hence the number of iterations to process and hence the execution time increases linearly.



Signature DNA Length

As the signature length increases, the number of character comparisons required increases, increasing the workload for each thread. However, longer signatures also reduces the likelihood of matches, reducing the number of updates required to the best score being tracked, allowing threads to terminate earlier. Furthermore, as the signature length increases, the number of starting positions of the sliding window ($\text{sample_len} - \text{signature_len} + 1$) decreases, reducing the number of iterations required. As such, as signature length increases, execution time remains largely constant with a slight decrease.



Number of signatures and Number of samples

As the number of signatures to be checked against increases, the number of blocks in the y dimension of the CUDA grid for the `matchKernel` increases since each block in the y dimension corresponds to a specific signature. Likewise, as the number of samples to check increases, the number of blocks in the x dimension of the CUDA grid for the `matchKernel` increases since each block in the x dimension corresponds to a specific sample.

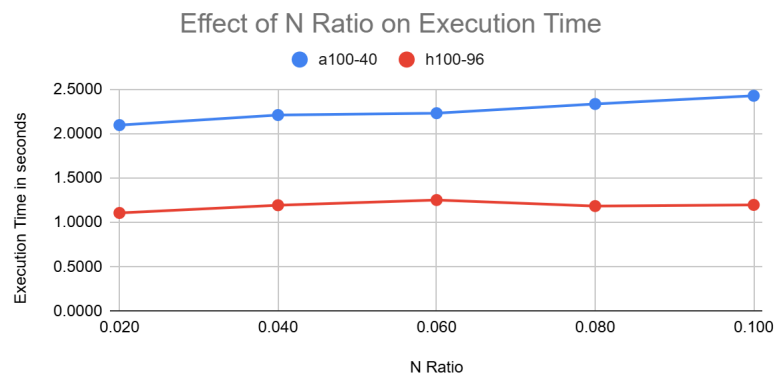
As such, the number of computations and hence execution time increases linearly with the number of signatures and number of samples.



Percentage of 'N' nucleotides

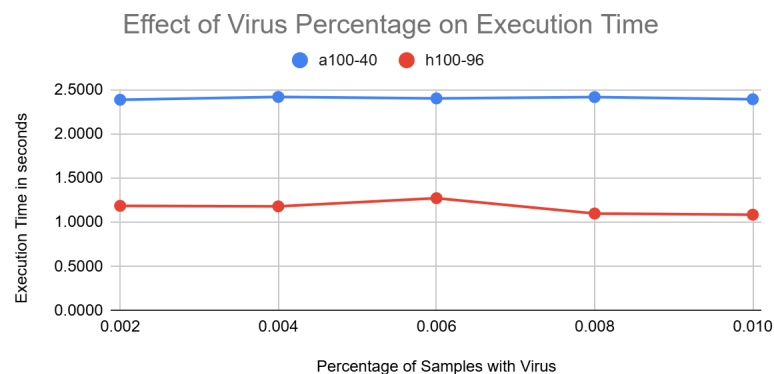
As the percentage of 'N' nucleotides increases, the frequency of mismatches decreases, which should result in fewer early terminations and more complete iterations, resulting in execution time increasing. However, execution time actually remains constant since CUDA threads within a warp execute instructions in lockstep (i.e. simultaneously on different data), otherwise known as SIMD (Single Instruction, Multiple Data) execution. As such, even if some threads in the warp encounter a mismatch that allows them to terminate the check early, the warp already must wait for all threads to complete their checks fully. Threads that

terminate early are simply masked out (inactive) but the warp as a whole continues to execute. Thus, even if a higher percentage of 'N' nucleotides causes fewer threads to terminate early, the runtime of the warp remains largely constant since the remaining threads in the warp still need to execute fully. The presence of more 'N' nucleotides only increases the average amount of work done by individual threads but not the overall runtime of the warp. As such, the runtime of the kernel is determined by the slowest warp in the grid. Since all warps execute in lockstep and must process the same number of sliding window positions, execution time is not affected significantly by the percentage of 'N' nucleotides.



Percentage of samples with (any no. of) viruses

The percentage of samples with viruses (i.e. samples that match one or more signatures) does not significantly affect execution time as the runtime of the CUDA kernel is primarily determined by the number of threads, blocks, and the amount of work each thread performs, rather than the actual results of the comparisons. The only additional cost is in the post-kernel processing on the CPU, which is negligible compared to the GPU runtime.



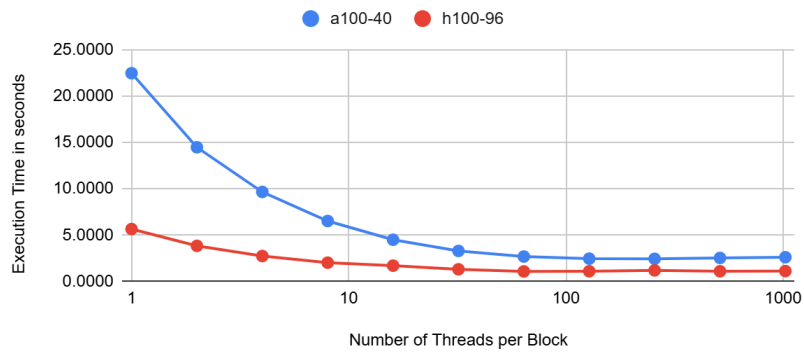
Threads per block

matchKernel1

Increasing the number of threads per block initially results in a quadratic reduction in execution time. Since no writes are made to the sample and signature data, sample-signature matches are independent and can be efficiently parallelized so the performance speedup per thread significantly enhances execution speed. Since threads are executed in warps (32 threads per warp). If the number of threads per block is too low (e.g. less than 32), some threads in the last warp may remain idle, resulting in warp underutilization and hence lower overall throughput. However, performance plateaus beyond

a certain point due to resource limitations, such as shared memory congestion, thread scheduling overhead, or insufficient parallel workload per thread.

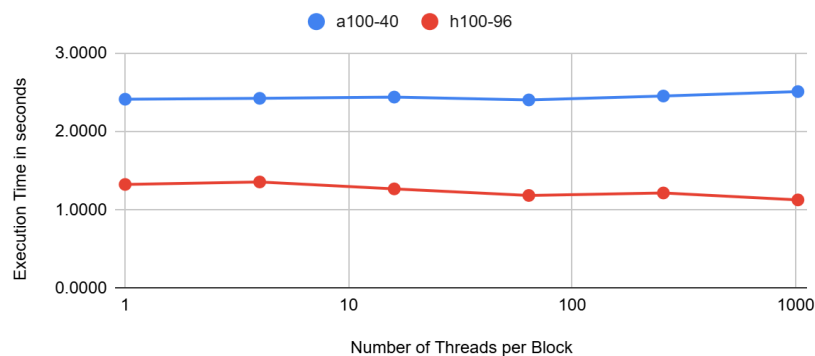
Effect of Threads per Block (Match Kernel) on Execution Time



integrityHashKernel

However, a similar speedup for `integrityHashKernel` is not observed since the integrity hash is calculated per sample. The total workload is proportional to the number of samples, not the number of threads per block. Each thread processes one sample independently so the runtime is determined by the time taken for all threads to complete. The number of threads per block only determines how the threads are grouped into blocks, but does not change the total number of threads launched. Since the total number of threads remains constant, the total workload and runtime remain constant, regardless of how the threads are grouped into blocks. Furthermore, most of the warp stalls were due to memory access in `matchKernel` (See [Appendix C](#)). The `integrityHashKernel` is less intensive than `matchKernel`, resulting in negligible performance changes when changing the number of threads per block in `integrityHashKernel`.

Effect of Threads per Block (Hash Kernel) on Execution Time



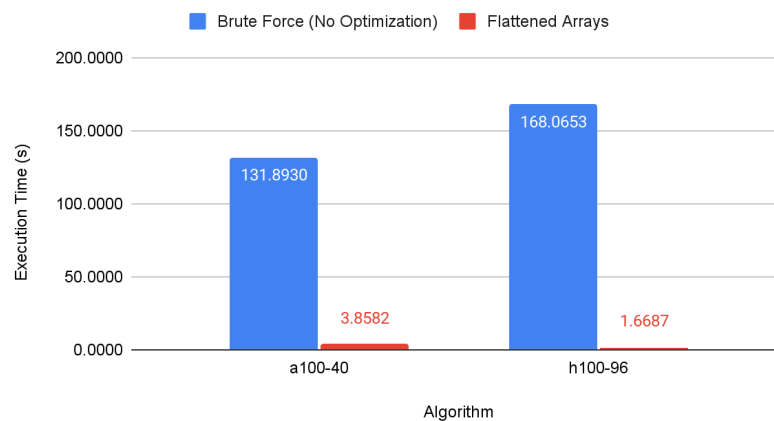
Optimizations

Flattening of Data Structures

In our 1st implementation (`kernel_skeleton_v1.cu`), the sample and signature sequences and quality scores were stored separately as strings (i.e. char arrays) within each sample/signature struct, leading to inefficient memory access patterns. As such, in our 2nd implementation (`kernel_skeleton_v2.cu`), the sequences and quality scores were flattened into a single contiguous array (`flat_sample_seqs`, `flat_sample_qual`s, and

`flat_signatures`) with offsets stored in separate arrays (`sample_offsets` and `signature_offsets`) used to access each sample/signature's sequence or quality scores. Flattening the data ensures that threads in a warp access contiguous memory locations, enabling coalesced memory access and reducing global memory latency. Flattened arrays also reduce memory overhead by eliminating the need for per-sample memory allocations, reducing memory fragmentation.

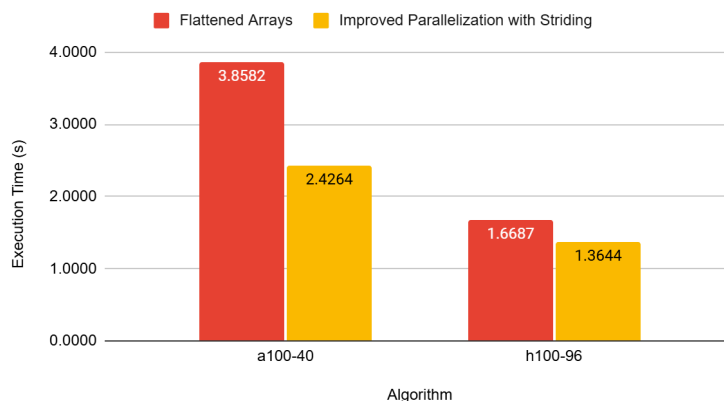
Effect of Optimization on Execution Time



Thread-Level Parallelisation for Sample-Signature Character Comparison

In our 2nd implementation (`kernel_skeleton_v2.cu`), each thread processed all sliding window positions for its assigned sample-signature pair sequentially. As such, in our 3rd implementation (`kernel_skeleton_v3.cu`), threads within a block processed sliding window positions in parallel using a stride-based approach. Each thread processed a subset of positions, reducing the workload per thread. By increasing parallelism by distributing the sliding window positions across threads, the workload would be better balanced and more computations would be performed in parallel. The stride-based approach also ensures that all threads in a warp execute similar instructions, minimising warp divergence. Since threads within a block collaborate to find matches and compute the best score, this improves warp efficiency.

Effect of Optimization on Execution Time



Acknowledgements

ChatGPT and GitHub Copilot were used to assist in generating bash scripts for ease of testing.

Appendix

Appendix A: Code Implementation

High-level overview of steps taken by the program:

1. Flatten the sample and signature data into separate arrays containing the sample sequences, sample quality scores, signature sequences.
2. Allocate GPU memory with `cudaMalloc`
3. Copy data to GPU with `cudaMemcpy`
4. Launch kernels (`matchKernel` and `integrityHashKernel`)
5. Copy results back to CPU with `cudaMemcpy`
6. Populate matches
7. Free memory with `cudaFree`

```

void runMatcher(const std::vector<klibpp::KSeq> &samples, const
std::vector<klibpp::KSeq> &signatures, std::vector<MatchResult> &matches)
{
    // Flatten samples and signatures
    ...

    // Allocate GPU memory with cudaMalloc
    cudaMalloc(...);
    ...

    // Copy data to GPU with cudaMemcpy
    cudaMemcpy(..., cudaMemcpyHostToDevice);
    ...

    // Launch kernels
    matchKernel<<<blocksPerGrid, threadsPerBlock>>>(...);
    integrityHashKernel<<<hashBlocksPerGrid, hashThreadsPerBlock>>>(...);

    // Copy results back to CPU with cudaMemcpy
    cudaMemcpy(..., cudaMemcpyDeviceToHost);
    ...

    // Populate matches
    for (int i = 0; i < num_samples; i++)
    {
        for (int j = 0; j < num_signatures; j++)
        {
            if (matchScores[i * num_signatures + j] > 0.0)
            {
                MatchResult result;
                result.sample_name = samples[i].name;
                result.signature_name = signatures[j].name;
                result.match_score = matchScores[i * num_signatures + j];
                result.integrity_hash = integrityHashes[i];
                matches.push_back(result);
            }
        }
    }

    // Free GPU memory with cudaFree
    cudaFree(...);
    ...
}

```

Appendix B: Performance Analysis

The spreadsheet with the processed data can be found [here](#).

All tests were run on xgph10 (a100-40) or xgpi1 (h100-96).

Variables:

Measurements for the following variables were taken. The ranges for the variables measured are given below (If the variables are not being tested, they are kept at the constant value specified unless otherwise mentioned):

Input parameters

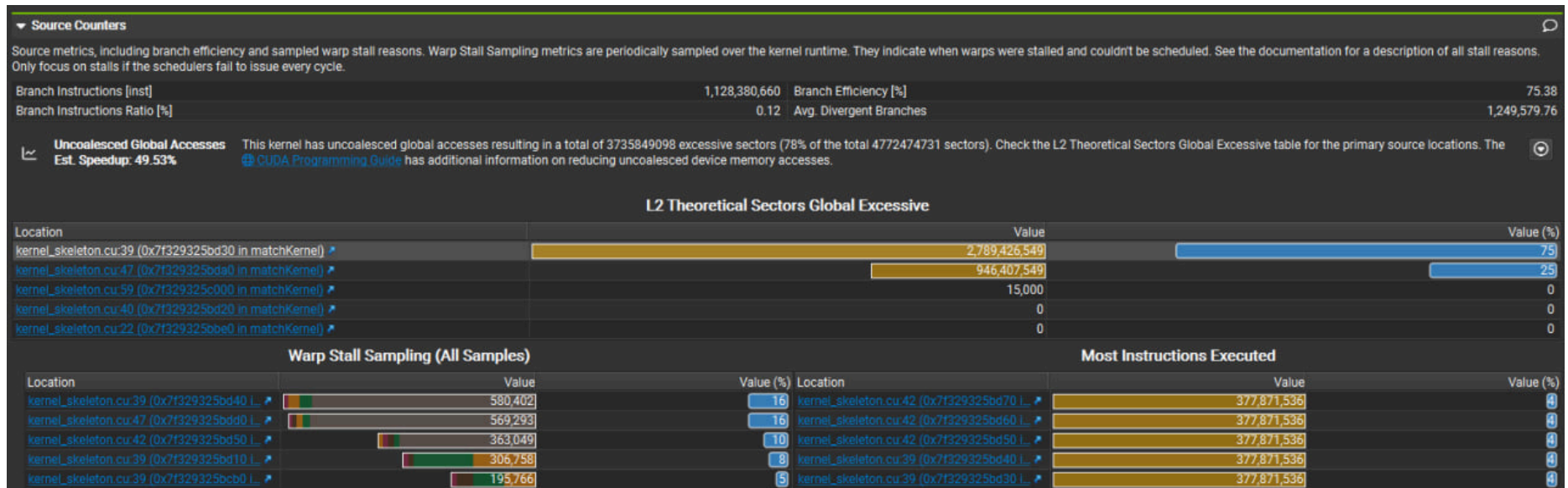
- Number of signatures: [500, 1000] (Constant: 1000)
 - 500, 600, 700, 800, 900, 1000
- Signature DNA Length: [3000, 10000] (Constant: 10000)
 - 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000
- Number of samples: [1000, 2200] (Constant: 1000)
 - 1000, 1200, 1400, 1600, 1800, 2000, 2200
- Sample DNA Length: [100000, 2000000] (Constant: 100000)
 - 100,000, 120,000, 140,000, 160,000, 180,000, 200,000
- Percentage of 'N' nucleotides: [0, 0.1] (Constant: 0.1)
 - 0.02, 0.04, 0.06, 0.08, 0.10
- Percentage of samples with (any no. of) viruses: [0, 0.01] (Constant: 0.01)
 - 0.002, 0.004, 0.006, 0.008, 0.010
- Number of viruses: [1, 5] for all test cases
- Phred: [10, 30] for all test cases

CUDA parameters

- Threads per block (for Match Kernel): [1, 1024] (Constant: 256)
 - 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024
- Threads per block (for Hash Kernel): [1, 1024] (Constant: 256)
 - 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024

Appendix C: Profiling

As seen from below, a large amount of warp stalls were from the specified lines of code which dealt with accessing the sample and signature characters. This shows that most of the stalls were due to the matcher kernel rather than the hash kernel.



Appendix D: Raw Data

Results from Optimizations Made

	Averaged Data		a100-40			h100-96		
Algorithm	a100-40	h100-96	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time 1	Execution Time 2	Execution Time 3
Brute Force (No Optimization)	131.8930	168.0653	131.8570	131.8090	132.0130	168.1080	168.0380	168.0500
Flattened Arrays	3.8582	1.6687	3.8578	3.8841	3.8327	1.6763	1.6652	1.6647
Improved Parallelization with Striding	2.4264	1.3644	2.4394	2.4069	2.4328	1.3928	1.3588	1.3415

Results from varying Signature Data Length

	A100					H100			
Signature DNA Length	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)	Signature DNA Length	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)
3000	2.64245	2.56304	2.56227	2.5893	3000	1.15584	1.11881	1.42366	1.2328
4000	2.55988	2.54326	2.56420	2.5558	4000	1.14754	1.12584	1.41825	1.2305
5000	2.57013	2.55887	2.55366	2.5609	5000	1.15014	1.11497	1.40955	1.2249
6000	2.52851	2.51781	2.53491	2.5271	6000	1.14425	1.10775	1.39921	1.2171
7000	2.50861	2.49372	2.49359	2.4986	7000	1.16336	1.13712	1.14042	1.1470
8000	2.49167	2.48407	2.49493	2.4902	8000	1.13468	1.09823	1.38384	1.2056
9000	2.44167	2.45799	2.45006	2.4499	9000	1.12187	1.08535	1.37224	1.1932
10000	2.56900	2.44006	2.43357	2.4809	10000	1.09064	1.10451	1.35852	1.1846

Results from varying Sample Data Length

	A100					H100			
Sample DNA Length	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)	Sample DNA Length	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)
100,000	2.42261	2.41767	2.41856	2.4196	100,000	1.15302	1.60000	1.10981	1.2876
120,000	2.86590	2.91496	3.02403	2.9350	120,000	1.28654	1.66670	1.27354	1.4089
140,000	3.36161	3.38575	3.48299	3.4101	140,000	1.45385	1.82749	1.44245	1.5746
160,000	3.96251	3.90685	3.98174	3.9504	160,000	1.63279	2.08139	1.57877	1.7643
180,000	4.43015	4.45476	4.39707	4.4273	180,000	1.75177	2.28549	2.37600	2.1378
200,000	4.87321	4.89832	4.84172	4.8711	200,000	1.92958	1.88851	2.51619	2.1114

Results from varying Number of Signatures

	A100					H100			
Number of Signatures	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)	Number of Signatures	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)
500	1.44240	1.45765	1.41713	1.4391	500	0.95290	0.96052	0.95868	0.9574
600	1.62726	1.65935	1.61138	1.6327	600	0.86786	1.03287	1.02612	0.9756
700	1.82861	1.80190	1.82617	1.8189	700	1.11068	1.13733	1.10832	1.1188
800	2.03093	2.03519	2.00539	2.0238	800	1.19283	0.98067	0.99659	1.0567
900	2.24569	2.21082	2.23541	2.2306	900	1.29876	1.28345	1.28710	1.2898
1,000	2.42392	2.43351	2.43251	2.4300	1,000	1.36771	1.39138	1.36893	1.3760

Results from varying Number of Samples

	A100					H100			
Number of Samples	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)	Number of Samples	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)
1,000	2.41966	2.42257	2.42900	2.4237	1,000	1.11657	1.34618	1.35224	1.2717
1,200	2.92652	2.96223	2.94289	2.9439	1,200	1.63769	1.64680	1.35193	1.5455
1,400	3.36400	3.41369	3.39031	3.3893	1,400	1.84226	1.85462	1.83369	1.8435
1,600	3.77109	3.76222	3.76222	3.7652	1,600	1.62648	1.59557	1.59580	1.6060
1,800	4.17489	4.18327	4.18287	4.1803	1,800	2.55131	1.74347	2.25988	2.1849
2,000	4.61323	4.61574	4.62020	4.6164	2,000	2.49955	1.85422	1.85845	2.0707
2,200	5.24485	5.24266	5.24940	5.2456	2,200	2.16835	2.22531	2.18045	2.1914

Results from varying N Ratio

	A100					H100			
N Ratio	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)	N Ratio	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)
0.020	2.08607	2.10513	2.09682	2.0960	0.020	1.21129	1.05135	1.05784	1.1068
0.040	2.38507	2.15663	2.08607	2.2093	0.040	1.25931	1.26113	1.06557	1.1953
0.060	2.22415	2.23077	2.23563	2.2302	0.060	1.26524	1.41588	1.08015	1.2538
0.080	2.32382	2.33879	2.33814	2.3336	0.080	1.33707	1.10409	1.11577	1.1856
0.100	2.43221	2.41456	2.43408	2.4270	0.100	1.10125	1.37374	1.12225	1.1991

Results from varying Percentage of Samples with Virus

	A100					H100			
Percentag e of Samples with Virus	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)	Percentag e of Samples with Virus	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)
0.002	2.38053	2.38509	2.39174	2.3858	0.002	1.09603	1.36304	1.10229	1.1871
0.004	2.40061	2.42595	2.43191	2.4195	0.004	1.35734	1.09657	1.08834	1.1808
0.006	2.41371	2.39870	2.39216	2.4015	0.006	1.37213	1.09002	1.35864	1.2736
0.008	2.41646	2.41732	2.39242	2.4087	0.008	1.10099	1.10320	1.09238	1.0989
0.010	2.39251	2.49283	2.32183	2.4024	0.010	1.08721	1.21039	1.08923	1.1289

Results from varying Threads per Block on Integrity Hash Kernel

	A100					H100			
Number of Threads per Block	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)	Number of Threads per Block	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)
1	2.4126	2.4103	2.4079	2.4103	1	1.3251	1.3219	1.3260	1.3243
4	2.4243	2.4148	2.4264	2.4218	4	1.3497	1.3633	1.3549	1.3559
16	2.4566	2.4308	2.4291	2.4389	16	1.3554	1.0923	1.3585	1.2687
64	2.4054	2.4030	2.3951	2.4012	64	1.0915	1.0879	1.3710	1.1835
256	2.4055	2.4499	2.4975	2.4510	256	1.1210	1.4037	1.1219	1.2155
1024	2.5419	2.5493	2.4349	2.5087	1024	1.1116	1.1329	1.1362	1.1269

Results from varying Threads per Block on Matcher Kernel

	A100					H100			
Number of Threads per Block	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)	Number of Threads per Block	Execution Time 1	Execution Time 2	Execution Time 3	Execution Time (Average)
1	2.4126	2.4103	2.4079	2.4103	1	1.3251	1.3219	1.3260	1.3243
4	2.4243	2.4148	2.4264	2.4218	4	1.3497	1.3633	1.3549	1.3559
16	2.4566	2.4308	2.4291	2.4389	16	1.3554	1.0923	1.3585	1.2687
64	2.4054	2.4030	2.3951	2.4012	64	1.0915	1.0879	1.3710	1.1835
256	2.4055	2.4499	2.4975	2.4510	256	1.1210	1.4037	1.1219	1.2155
1024	2.5419	2.5493	2.4349	2.5087	1024	1.1116	1.1329	1.1362	1.1269